

实验二：卷积神经网络（CNN）

▼ 实验二：卷积神经网络（CNN）

▼ 概述

实现内容

你将运行什么

环境配置

任务总览与推荐完成顺序

▼ Task 0：全连接层（仿射、ReLU 与 softmax）

步骤 0.1： `affine_forward`

步骤 0.2： `affine_backward`

步骤 0.3： `relu_forward` / `relu_backward`

步骤 0.4： `softmax_loss`

检查点 O

▼ Task 1：卷积层（`im2col` 前向与反向传播）

步骤 1.1： `conv_output_size`

步骤 1.2： `im2col`

步骤 1.3： `col2im`

步骤 1.4： `conv_forward_naive`

步骤 1.5： `conv_forward`

步骤 1.6： `conv_backward`

检查点 A

▼ Task 2：池化层、感受野与参数量

步骤 2.1： `max_pool_forward`

步骤 2.2： `max_pool_backward`

步骤 2.3： `receptive_field`

步骤 2.4： `count_conv_parameters`

检查点 B

▼ Task 3：完整 CNN 与 CIFAR-10 训练

3.1 数据集与预处理

▼ 3.2 组装 `ConvNet` 类

步骤 3.1： `_conv_relu_pool`

步骤 3.2： `_conv_relu_pool_backward`

步骤 3.3： `loss` 与 `_backward`

步骤 3.4： `predict`

▼ 3.3 训练循环（`src/train.py`）

步骤 4.1：模块级 `predict`

步骤 4.2： `evaluate`

步骤 4.3： `train`

检查点 D：全部通过

3.4 正式训练与调参（报告内容）

你在实验报告里面需要写什么？

提供的代码（无需修改）

▼ 评分标准

第一部分：自动化测试（80 分）

概述

上次实验里的全连接层有一个明显的浪费：它把输入当成一串互不相干的数字，完全不利用图像里「相邻像素相关」这个事实，而且参数量随输入尺寸线性增长——处理一张 32×32 的彩色图像，一个全连接层就需要上百万个参数。****卷积神经网络**（Convolutional Neural Network, CNN）******用两个想法解决了这个问题：

- 1. **局部连接**：每个输出只看输入的一小块区域（感受野）；
- 2. **权重共享**：同一组权重在整张图上滑动使用。

结果是参数量只取决于卷积核大小和通道数，与图像尺寸无关；权重共享还带来了**平移等变性**——同一个特征检测器在图像的任何位置都能生效。这一结构由 LeCun 等人在 LeNet-5 中奠定[1]，并在 AlexNet[2] 之后成为计算机视觉的主流。

本次实验的目标是：**只用 NumPy，从零实现一个完整的卷积神经网络，并在真实的 CIFAR-10 彩色图像 10 分类数据集上把它训练出来。**

实现内容

按依赖关系，你需要依次完成以下五个部分。**前面的部分是后面的地基，请务必按顺序完成，不要跳步：**

顺序	部分	内容	所在文件
①	Task 0	全连接层： affine_forward → affine_backward → relu_forward → relu_backward → softmax_loss	src/layers.py
②	Task 1	卷积层： conv_output_size → im2col → col2im → conv_forward_naive → conv_forward → conv_backward	src/im2col.py 、 src/conv.py
③	Task 2	池化层、感受野与参数量： max_pool_forward → max_pool_backward → receptive_field → count_conv_parameters	src/pool.py
④	Task 3a	组装 CNN： ConvNet 类（前向/反向/预测）	src/cnn.py
⑤	Task 3b	训练循环： predict → evaluate → train	src/train.py

完成①②③④⑤后，再在 CIFAR-10 上做健全性检查、正式训练与调参（报告内容）。

关于 Task 0（全连接层）：仿射层、ReLU 与 softmax 是本实验其余部分的公共积木（卷积的反向本质是仿射层的反向；组装 CNN 和 test_pool_cnn.py 的端到端测试都直接调用它们），但它们本身与卷积、池化互不依赖。你可以先做，也可以在 Task 1 卡住时切换过来「换换脑子」。

你将运行什么

- 1. 用 uv run pytest 跑测试（全部 47 例）；
- 2. 对每个函数跑**对应的单测**来即时验证（见各步骤末尾的「验证」小节）；
- 3. 用 uv run python train_cifar10.py 在 CIFAR-10 上训练完整模型；
- 4. 调节训练时的超参数，并做三组对照实验并写报告。

环境配置

请首先安装 uv Python 项目管理工具：

```
# On macOS and Linux.
curl -LsSf https://astral.sh/uv/install.sh | sh

# On Windows.
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

先确认安装完毕，然后在项目根目录下执行：

```
uv sync
```

该指令会在根目录下创建好 Python 虚拟环境。完成本实验所需的依赖仅存在于该虚拟环境中，若使用 IDE，可先将解释器设置为该环境中的解释器。

之后可以尝试运行测试用例（**路径中不可带非 ascii 编码字符**）：

```
uv run pytest
```

若输出类似以下的内容，则说明环境配置完毕：

```
..FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF [100%]
===== short test summary info =====
FAILED tests/test_layers.py::test_affine_forward_shapes_and_value - NotImplementedError
FAILED tests/test_conv.py::test_conv_output_size_formula - NotImplementedError
FAILED tests/test_conv.py::test_im2col_shape - NotImplementedError
...
44 failed, 3 passed in 0.25s
```

其中 3 个通过的测试只检查已提供的构造函数与配置类，属于预期行为；其余 44 个测试将在你完成相应任务后逐一变绿。

关于如何只跑一个测试：全程都用 `uv run pytest` 跑全部 47 个既慢又吵。更高效的做法是只跑你正在写的那个函数的测试，例如：

```
uv run pytest tests/test_layers.py           # 只跑全连接层的 7 个测试
uv run pytest tests/test_conv.py -k conv_output_size  # 只跑名字含 conv_output_size 的测试
uv run pytest tests/test_conv.py           # 只跑卷积的 14 个测试
```

其中 `-k` 后面跟的是测试函数名的**子串**。本文每个步骤末尾都会给出对应的命令。

任务总览与推荐完成顺序

下面这张表是整个实验的「地图」。**第 2 列「依赖」告诉你必须先完成什么**；请严格按照从上到下的顺序推进，每完成一行就做一次「检查点」确认，再进入下一行。

步骤	函数/方法	依赖	完成后跑什么验证	所在文件
0.1	affine_forward	无	test_affine_forward_shapes_and_value 等	src/layers.py
0.2	affine_backward	0.1	test_affine_backward_gradients	src/layers.py
0.3	relu_forward / relu_backward	无	test_relu_forward_and_backward 等	src/layers.py
0.4	softmax_loss	无	test_softmax_loss_value_and_gradient 等	src/layers.py
0	检查点 O：全连接层全部通过	—	uv run pytest tests/test_layers.py (7/7)	—
1.1	conv_output_size	无	test_conv_output_size_formula	src/im2col.py
1.2	im2col	1.1	4 个 im2col 测试	src/im2col.py
1.3	col2im	1.2	2 个 col2im 测试	src/im2col.py
1.4	conv_forward_naive	1.1	test_conv_forward_naive_hand_computed	src/conv.py
1.5	conv_forward	1.2、 1.4	test_conv_forward_matches_naive 等	src/conv.py
1.6	conv_backward	1.3、 1.5	test_conv_backward_gradients 等	src/conv.py
1	检查点 A：卷积层全部通过	—	uv run pytest tests/test_conv.py (14/14)	—
2.1	max_pool_forward	1.2	3 个 max_pool_forward 测试	src/pool.py
2.2	max_pool_backward	2.1	2 个 max_pool_backward 测试	src/pool.py
2.3	receptive_field	无 (可与 1.x 并行)	5 个 receptive_field 测试	src/pool.py
2.4	count_conv_parameters	无 (可与 1.x 并行)	test_conv_parameter_count_...	src/pool.py
2	检查点 B：池化+感受野全部通过	—	uv run pytest tests/test_pool_cnn.py (14/14)	—
3.1	ConvNet._conv_relu_pool	检查点 O、 A、B	(被 3.3 覆盖)	src/cnn.py
3.2	ConvNet._conv_relu_pool_backward	3.1	(被 3.3 覆盖)	src/cnn.py
3.3	ConvNet.loss / _backward	3.1、 3.2	test_gradients_match_numeric 等	src/cnn.py

步骤	函数/方法	依赖	完成后跑什么验证	所在文件
3.4	ConvNet.predict	3.3	test_forward_scores_shape	src/cnn.py
3	检查点 C：模型测试通过	—	uv run pytest tests/test_cnn_model.py (6/6)	—
4.1	predict (模块级)	3.4	test_predict_returns_labels	src/train.py
4.2	evaluate	4.1	test_evaluate_returns_loss_and_acc	src/train.py
4.3	train	4.2	test_train_reduces_loss 等	src/train.py
4	检查点 D：全部 47 例通过	—	uv run pytest (47/47)	—

说明：2.3、2.4 两个函数与卷积完全无关（纯公式推导），如果你在 Task 1 卡住了想换个脑子，可以随时跳到它们那里「换换口味」，不影响依赖关系。

Task 0：全连接层（仿射、ReLU 与 softmax）

在卷积之前，先把「仿射（全连接）层 + 激活函数 + 损失函数」这套最基础的积木搭好。这三个函数与图像、卷积、池化完全无关，却是后面一切的地基——conv_forward 的反向本质就是仿射层的反向，组装 ConvNet 和 test_pool_cnn.py 里的端到端测试也都直接调用它们。如果这是你的第一份实验，从这一节开始最合适；如果你之前写过 MLP，这一节应当几分钟就能做完。

这一整节只允许使用 NumPy，不得调用任何自动微分工具。

步骤 0.1：affine_forward

目标：实现仿射（全连接）层的前向 $out = xW^T + b$ ，其中 x 是展平后的输入。关键点在于：输入可能是任意高维的（例如卷积网络里展平前的 (N, C, H, W) ），前向时必须先把它展平成二维矩阵。

函数签名：

```
def affine_forward(x, w, b):
    # x: (N, d_1, ..., d_k), 内部展平为 (N, D), D = prod(d_i)
    # w: (D, M)
    # b: (M,)
    # 返回 (out: (N, M), cache: (x, w, b))
```

验证：

```
uv run pytest tests/test_layers.py -k affine_forward
```

应看到 2 passed。

步骤 0.2：affine_backward

目标：仿射层的反向。前向是 $out = XW^T + b$ ，其中 X 是 (N, D) 的展平输入，于是梯度为：

$$\partial X = (\partial out) W, \quad \partial W = (\partial out)^T X, \quad \partial b = \sum_n (\partial out)_n$$

函数签名：

```
def affine_backward(dout, cache):  
    # dout: (N, M) 上游梯度  
    # cache: (x, w, b), 来自 affine_forward  
    # 返回 (dx, dw, db)
```

验证：

```
uv run pytest tests/test_layers.py -k affine_backward
```

应看到 1 passed。

步骤 0.3: relu_forward / relu_backward

目标：实现 ReLU 激活函数及其反向。ReLU 是最常用的激活函数： $out = \max(0, x)$ ，反向时梯度只流向输入为正的位置。

函数签名：

```
def relu_forward(x):  
    # 返回 (out, cache)，其中 cache = x  
  
def relu_backward(dout, cache):  
    # cache: 来自 relu_forward (即输入 x)  
    # 返回 dx: 与 x 同形状
```

验证：

```
uv run pytest tests/test_layers.py -k relu
```

应看到 2 passed。

步骤 0.4: softmax_loss

目标：softmax + 交叉熵损失，以及损失对分数 scores 的梯度。这是分类任务的标准损失函数，也是后面 ConvNet.loss 直接调用的东西。

原理：

Softmax 函数将分数向量转换为概率分布。对于第 i 个样本的第 j 个类别，其概率为：

$$p_{ij} = \frac{e^{\text{scores}_{ij}}}{\sum_{k=0}^{C-1} e^{\text{scores}_{ik}}}$$

交叉熵损失衡量预测概率分布与真实标签之间的差异。对于 N 个样本，损失为所有样本负对数似然的平均值：

$$\text{loss} = -\frac{1}{N} \sum_{i=0}^{N-1} \log(p_{i,y_i})$$

其中 y_i 是第 i 个样本的真实类别标签。

反向传播时，损失对分数 `scores` 的梯度为：

$$\frac{\partial loss}{\partial scores_{ij}} = p_{ij} - \mathbf{1}_{j=y_i}$$

即对于第 i 个样本，真实类别对应的梯度为 $p_{i,y_i} - 1$ ，其他类别的梯度为 p_{ij} 。

函数签名：

```
def softmax_loss(scores, y):  
    # scores: (N, C)  
    # y: (N,) 整数标签, 取值 [0, C)  
    # 返回 (loss: float, dscores: (N, C))
```

验证：

```
uv run pytest tests/test_layers.py -k softmax_loss
```

应看到 2 passed。

检查点 0

完成以上 5 个函数后，跑：

```
uv run pytest tests/test_layers.py
```

应看到 **7 passed**。全连接层是卷积反向与 CNN 组装都要复用的积木。

Task 1：卷积层（im2col 前向与反向传播）

朴素的卷积实现需要四重循环，慢到无法使用。工程上的标准做法是 **im2col**：把每个卷积窗口展开成一行，整个卷积就变成一次矩阵乘法，可以直接调用高度优化的矩阵运算库（NumPy 底层的 BLAS）。其反向传播需要一个逆操作 **col2im**。

设输入 x 的形状为 (N, C, H, W) (N 张 C 通道、 $H \times W$ 的图像)，卷积核 w 的形状为 (F, C, HH, WW) (F 个卷积核)，步长 `stride`、填充 `pad`。则输出空间尺寸为：

$$H' = 1 + \frac{H + 2 \cdot \text{pad} - HH}{\text{stride}}, \quad W' = 1 + \frac{W + 2 \cdot \text{pad} - WW}{\text{stride}}$$

这一整节只允许使用 NumPy，不得调用任何自动微分工具。

步骤 1.1：conv_output_size

目标：写出输出空间尺寸公式，这是后续所有函数的地基。

函数签名：

```
def conv_output_size(H, W, HH, WW, stride, pad) -> tuple[int, int]:  
    # 返回 (out_h, out_w)
```

验证：

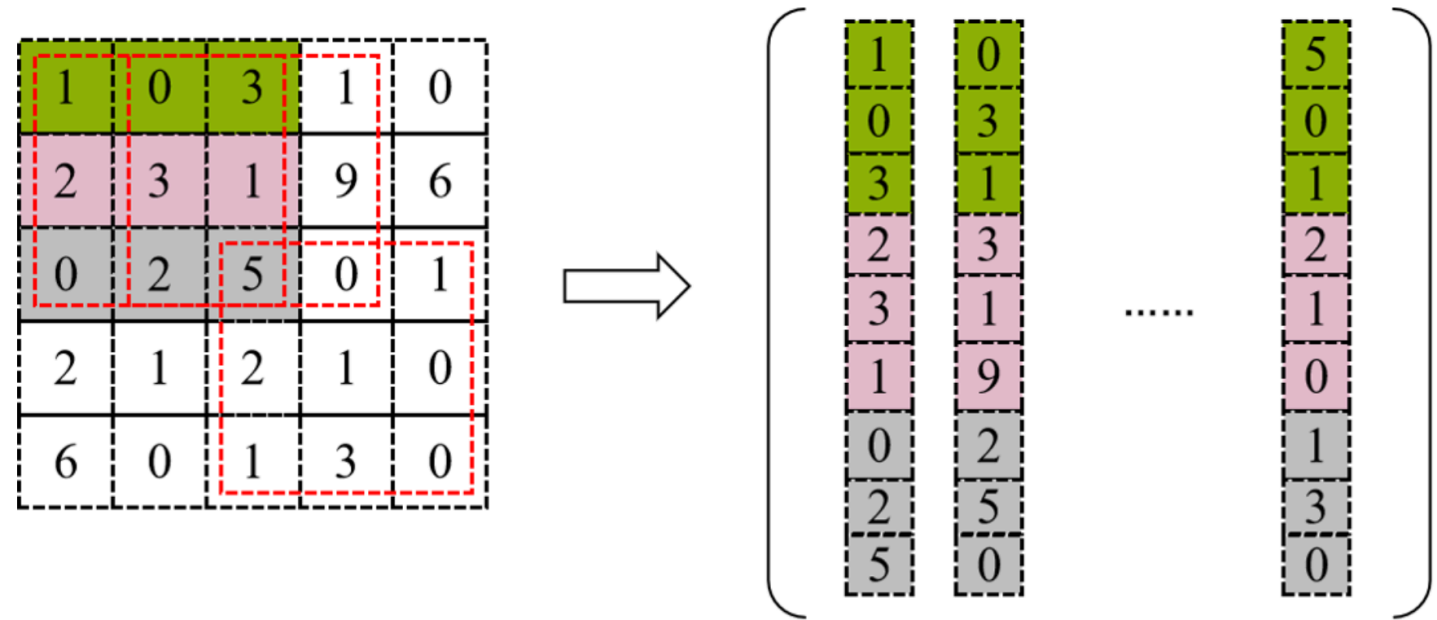
```
uv run pytest tests/test_conv.py -k conv_output_size
```

应看到 1 passed。

步骤 1.2： im2col

目标：把「对每个输出位置取一块 patch」这件事改写成「逐核位置切片」，从而把卷积变成一次矩阵乘法。这是全实验最核心的一个函数。

原理：Im2Col 算法的优化策略为用空间换时间，用连续的行向量的存储空间作为代价优化潜在的内存访问消耗的时间。其实施过程依据的是线性代数中最基本的矩阵运算的原理。使用 Im2Col 将输入矩阵展开一个大矩阵，矩阵每一列表示卷积核需要的一个输入数据，按行向量方式存储。如图所示：



函数签名：

```
def im2col(x, HH, WW, stride, pad):  
    # x: (N, C, H, W)  
    # 返回 col: (N * out_h * out_w, C * HH * WW)
```

验证：

```
uv run pytest tests/test_conv.py -k im2col
```

应看到 4 passed (shape 、 extracts_correct_patch 、 applies_zero_padding 、 matches_manual_slicing)。

步骤 1.3： col2im

目标：实现 im2col 在反向传播意义下的「逆」操作——把 patch 矩阵的梯度折回图像布局。

函数签名：

```
def col2im(col, x_shape, HH, WW, stride, pad):
    # col: (N * out_h * out_w, C * HH * WW)
    # x_shape: 原始输入的形状 (N, C, H, W)
    # 返回 x: (N, C, H, W)
```

验证：

```
uv run pytest tests/test_conv.py -k col2im
```

应看到 2 passed。

步骤 1.4： conv_forward_naive

目标：用最直白的四重循环实现卷积。它不追求速度，唯一使命是作为「显然正确」的基准，供 conv_forward 比对。只在测试的小输入上用它，训练时绝不调用。

原理：

卷积操作的数学定义是：对每个输出位置，将卷积核与输入对应窗口做逐元素乘积累加，再加上偏置。

对于输入 x 形状为 (N, C, H, W) ，卷积核 w 形状为 (F, C, HH, WW) ，偏置 b 形状为 $(F,)$ ，输出特征图 out 形状为 (N, F, H_{out}, W_{out}) ，其中：

$$H_{out} = \frac{H + 2 \cdot pad - HH}{stride} + 1$$

$$W_{out} = \frac{W + 2 \cdot pad - WW}{stride} + 1$$

单个输出位置的计算公式为：

$$out[n, f, i, j] = b[f] + \sum_{c=0}^{C-1} \sum_{hh=0}^{HH-1} \sum_{ww=0}^{WW-1} x_{pad}[n, c, i \cdot stride + hh, j \cdot stride + ww] \cdot w[f, c, hh, ww]$$

函数签名：

```
def conv_forward_naive(x, w, b, conv_param):
    # x: (N, C, H, W), w: (F, C, HH, WW), b: (F,)
    # conv_param: {"stride": int, "pad": int}
    # 返回 (out: (N, F, out_h, out_w), cache)
```

验证：

```
uv run pytest tests/test_conv.py -k conv_forward_naive_hand_computed
```

应看到 1 passed。

步骤 1.5： conv_forward

目标：用 im2col + 一次矩阵乘法实现卷积，结果与朴素版数值完全一致，但快几个数量级。

签名：

```
def conv_forward(x, w, b, conv_param):  
    # 返回 (out: (N, F, out_h, out_w), cache)
```

验证：

```
uv run pytest tests/test_conv.py -k "conv_forward_matches_naive or conv_forward_shapes or conv_bias_shifts_each_filter or
```

应看到 4 passed。

步骤 1.6： conv_backward

目标：完成卷积的反向传播。既然前向是 $\text{col} @ \text{w_col.T} + \text{b}$ ，反向就是仿射层的那一对矩阵乘法，唯一额外的活是把 patch 矩阵的梯度用 `col2im` 折回图像布局。

函数签名：

```
def conv_backward(dout, cache):  
    # dout: (N, F, out_h, out_w)  
    # cache: 来自 conv_forward  
    # 返回 (dx, dw, db)
```

验证：

```
uv run pytest tests/test_conv.py -k conv_backward
```

应看到 2 passed。

检查点 A

完成以上 6 个函数后，跑：

```
uv run pytest tests/test_conv.py
```

应看到 **14 passed**。如果还没全绿，先别进入后续的任务——池化和组装 CNN 都建立在这些积木之上。

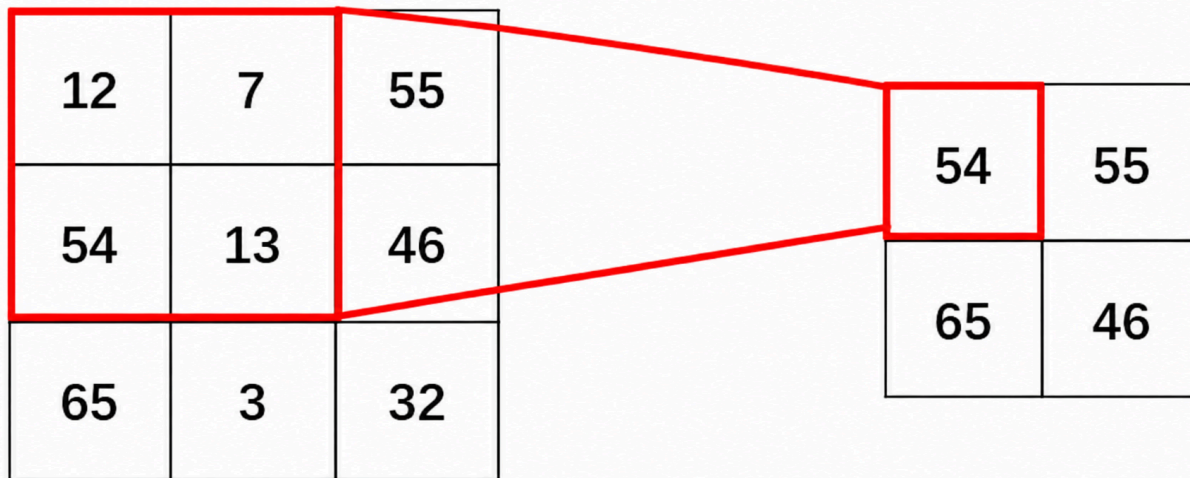
Task 2：池化层、感受野与参数量

最大池化（max pooling）在不引入任何参数的前提下缩小特征图并提供一定的位置不变性。它没有参数，反向传播时把梯度全部路由给窗口内的最大值，其余位置梯度为零。

步骤 2.1： max_pool_forward

目标：用 `im2col` 实现最大池化前向。关键技巧是——池化**逐通道进行**，所以先把通道维「折进」batch 维，问题就退化成单通道的池化。

原理：池化操作也是原图像矩阵与一个固定形状的窗口进行计算，并输出特征图的一种计算方式，卷积操作的卷积核是有数据（权重）的，而池化直接计算池化窗口内的原始数据，池化分为最大池化、平均池化等，最大池化示意图如下：



函数签名：

```
def max_pool_forward(x, pool_param):  
    # x: (N, C, H, W)  
    # pool_param: {"pool_height": PH, "pool_width": PW, "stride": stride}  
    # 返回 (out: (N, C, out_h, out_w), cache)
```

验证：

```
uv run pytest tests/test_pool_cnn.py -k "max_pool_forward or max_pool_is_per_channel or max_pool_output_shapes"
```

应看到 3 passed。

步骤 2.2： max_pool_backward

目标：池化的反向——梯度**全部**给窗口内最大值所在位置，其余位置给 0。

函数签名：

```
def max_pool_backward(dout, cache):  
    # cache: 来自 max_pool_forward  
    # 返回 dx: 与池化输入同形状 (N, C, H, W)
```

验证：

```
uv run pytest tests/test_pool_cnn.py -k max_pool_backward
```

应看到 2 passed。

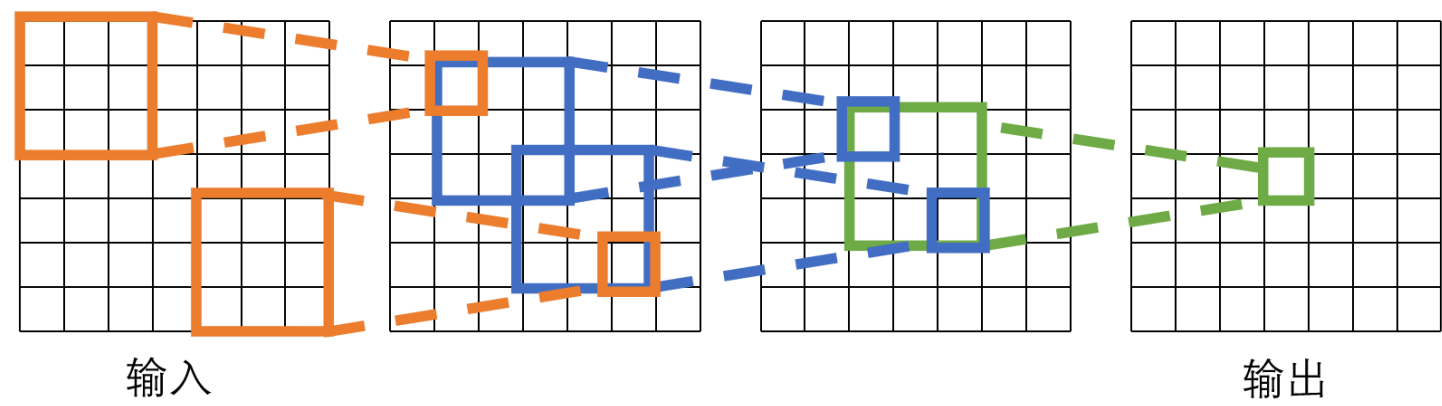
步骤 2.3： receptive_field

目标：计算堆叠网络的感受野与累计步长。多层卷积与池化叠加后，单个输出单元能看到的输入区域称为**感受野**（receptive field）。

原理：

感受野的大小从输出往输入方向递推：

$$r_{l-1} = (r_l - 1) \cdot \text{stride}_l + \text{kernel}_l, \quad r_{\text{输出}} = 1$$



累计步长 (jump) 从输入层到输出层的前向递推：

$$\text{jump}_l = \text{jump}_{l-1} \times \text{stride}_l$$

函数签名：

```
def receptive_field(layers) -> tuple[int, int]:
    # layers: [(kernel, stride), ...] 按前向顺序给出
    # 返回 (r, jump), jump 为累计步长
```

验证：

```
uv run pytest tests/test_pool_cnn.py -k receptive_field
```

应看到 5 passed。

步骤 2.4： count_conv_parameters

目标：计算一个卷积层的可学习参数量，体会「与输入尺寸无关」这一权重共享的意义。

原理：

$$\text{Total_Params} = (C_{in} \times C_{out} \times K \times K) + (B \times C_{out})$$

函数签名：

```
def count_conv_parameters(in_channels, out_channels, kernel_size, use_bias=True) -> int:
```

验证：

```
uv run pytest tests/test_pool_cnn.py -k conv_parameter_count
```

应看到 1 passed。

检查点 B

跑：

```
uv run pytest tests/test_pool_cnn.py
```

应看到 **14 passed**。其中最后两个（`test_cnn_end_to_end_gradients`、`test_cnn_learns_a_simple_visual_pattern`）已经把 Task 0、1、2 的积木拼成一个小的 conv-relu-pool-affine 网络做端到端验证——它们变绿，说明你已经具备组装完整 CNN 的全部零件了。

Task 3：完整 CNN 与 CIFAR-10 训练

前面几个任务搭好了积木，本任务用它们在真实数据上训练一个完整的模型。

3.1 数据集与预处理

CIFAR-10（见参考资料[3]）是计算机视觉最常用的入门数据集：50000 张 32×32 的彩色训练图像与 10000 张测试图像，共 10 个类别（飞机、汽车、鸟、猫、鹿、狗、蛙、马、船、卡车）。加载代码已经提供在 `src/data.py` 中。

关于数据集下载：CIFAR-10 的官方地址（www.cs.toronto.edu）在国内访问极慢，本实验已改用华为云 OBS 镜像（MindSpore 官方整理的二进制格式）：

```
https://ascend-professional-construction-dataset.obs.cn-north-4.myhuaweicloud.com/ComputerVision/cifar10_mindspore.z
```

`load_cifar10` 首次运行时会**自动下载该 zip 并解压**到 `data/cifar10_mindspore/`，再解析成 `data/cifar10.npz` 缓存，之后每次加载直接从缓存读取、瞬时完成。解压后的目录结构为：

```
data/cifar10_mindspore/
  data/10-batches-bin/  data_batch_1.bin ... data_batch_5.bin  # 50000 训练样本（二进制格式）
  data/10-verify-bin/   test_batch.bin          # 10000 测试样本
  images/ results/      （MindSpore 示例附带内容，本实验用不到）
```

如果你已经手动下载并解压好了这份数据，只要保证它位于 `data/cifar10_mindspore/` 即可，脚本会自动检测、跳过下载。加载与划分的用法如下：

```
from src.data import load_cifar10, train_val_split

X_train_full, y_train_full, X_test, y_test = load_cifar10("data")  # (50000,3,32,32) in [0,1]
X_tr, y_tr, X_val, y_val = train_val_split(X_train_full, y_train_full, num_val=5000)
```

注意：本任务的两个测试文件 `test_cnn_model.py` 与 `test_train.py` 全部在**合成数据或小随机输入**上运行，**不需要下载 CIFAR-10 即可通过全部测试**。你可以先写完代码、跑绿全部测试，再去下载真实数据集做正式训练。

3.2 组装 ConvNet 类

请在 `src/cnn.py` 中补全 `ConvNet` 类。**构造函数 `__init__` 已经写好**（这就是那 3 个天然通过的测试之一），你只需要补全**前向/反向相关的方法**。结构是最经典的 conv-pool 块堆叠（其中的 ReLU、仿射、softmax 直接复用你在 Task 0 里实现好的函数）：

```
[conv3x3(s1,p1) - ReLU - maxpool2x2(s2)] × num_blocks → flatten → affine → scores
```

- 每个 conv 用 3×3 核、stride 1、pad 1，保持空间尺寸不变；每个 pool 把尺寸减半。num_blocks=2、base_filters=32 时，32×32 的输入经两块后变为 64 通道 8×8，展平得 4096 维，映射到 10 类分数。
- 通道数逐块翻倍（32 → 64 → ...），这是 VGG 以来的惯例：空间越小、通道越多，计算量保持均衡。
- 参数以 dict 存储（w0/b0/w1/b1/.../wfc/bfc），梯度同样以 dict 返回——这样 optim.py 中的 sgd / sgd_momentum / adam 可以直接逐参数套用，无需任何修改。
- 卷积权重初始化用 src/init.py 中已提供的 he_conv（He 初始化在卷积上的推广， $\text{fan_in} = C \cdot k \cdot k$ ）。

步骤 3.1：_conv_relu_pool

函数签名：

```
def _conv_relu_pool(self, x, idx):  
    # 一个 conv-relu-pool 块的前向  
    # 返回 (out, (conv_cache, relu_cache, pool_cache))
```

实现要求：依次调用 conv_forward → relu_forward → max_pool_forward，把三个 cache 打包返回。注意卷积核大小为 3、步长 1、pad 1；池化核为 2、步长 2。

步骤 3.2：_conv_relu_pool_backward

函数签名：

```
def _conv_relu_pool_backward(self, dout, caches):  
    # caches = (conv_cache, relu_cache, pool_cache)  
    # 返回 (dx, dW, dB)
```

实现要求：按 max_pool_backward → relu_backward → conv_backward 的**逆序**反传，返回该块的 dx, dW, dB。

步骤 3.3：loss 与 _backward

函数签名：

```
def loss(self, X, y=None):  
    # X: (N, 3, 32, 32)，已归一化；y: (N,) 整数标签  
    # y=None → 返回 scores (N, num_classes)  
    # 否则 → 返回 (loss, grads)  
  
def _backward(self, dscores, caches, fc_cache, side, C, x_shape):  
    # 从 dscores 反传回所有参数的梯度，返回 grads dict
```

步骤 3.4：predict

函数签名：

```
def predict(self, X, batch_size=200):  
    # 分 batch 返回 argmax 类别标签 (N,)
```

验证：

```
uv run pytest tests/test_cnn_model.py
```

应看到 **6 passed**。

3.3 训练循环（src/train.py）

TrainConfig dataclass 已给出（这就是另一个天然通过的测试），你只需补全三个函数。

步骤 4.1：模块级 predict

函数签名：

```
def predict(net, X, batch_size=200):  
    # 返回预测标签 (N,)
```

实现要求：与 ConvNet.predict 语义相同，可直接委托 net.predict(X, batch_size)，或自己分 batch 调 net.loss。测试 test_predict_returns_labels 检查返回形状、整数 dtype 与取值范围 [0, 10)。

步骤 4.2：evaluate

函数签名：

```
def evaluate(net, X, y, batch_size=200):  
    # 返回 (平均损失, 准确率)
```

实现要求：分 batch 调 net.loss(batch, y_batch) 累加损失、统计预测正确的个数，最后返回（总损失/批次数，正确数/样本数）。

注意平均损失的归一化——net.loss 返回的是该 batch 内的平均损失，跨 batch 求和后应除以批次数。测试

test_evaluate_returns_loss_and_acc 验证未训练模型在 10 类上的准确率约为 0.1。

步骤 4.3：train

函数签名：

```
def train(X, y, cfg: TrainConfig, X_val=None, y_val=None, verbose=True):  
    # 返回 (net, history)
```

实现要求：

1. 构造 ConvNet(num_blocks=cfg.num_blocks, base_filters=cfg.base_filters, seed=cfg.seed)，并为每个参数初始化一个优化器 config（{k: {"learning_rate": cfg.lr} for k in net.params}）。
2. 从 {"sgd": sgd, "sgd_momentum": sgd_momentum, "adam": adam} 中按 cfg.optimizer 选出更新函数。
3. minibatch 主循环
4. 每个 epoch 结束时用 evaluate 记录训练集指标；若给了验证集，也记录验证集指标，存入 history。history 是一个 dict，含列表 train_loss、train_acc，以及（有验证集时）val_loss、val_acc。
5. verbose=True 时打印每轮进度。

验证：

```
uv run pytest tests/test_train.py
```

应看到 **6 passed**。

检查点 D：全部通过

```
uv run pytest
```

应看到 **47 passed**。至此，你的 CNN 实现已经通过了所有自动化验证。

3.4 正式训练与调参（报告内容）

在通过以上的全部测试后，现在需要在真实的CIFAR-10数据集上训练CNN图像分类模型，数据加载的代码和训练代码均已写好（src/data.py, train_cifar10.py）

你可以先用默认配置跑一个epoch（跑之前建议在src/train.py中的train函数中每10个批次打印一次进度，方便观察训练过程）

```
uv run python train_cifar10.py --epochs 1
```

默认配置在纯 NumPy 实现下每个 epoch 约需几分钟到十几分钟（视机器而定），5个epoch后测试准确率通常在 **65%~68%** 附近。

训练脚本会自动完成以下「输出」工作，这些也是验收时你需要提交的产物：

1. **损失 / 准确率曲线**：自动调用 matplotlib 把 history 里的 train_loss、val_loss、train_acc、val_acc 画成两张图，保存为 outputs/loss_curve.png 与 outputs/accuracy_curve.png。
2. **测试集逐类准确率**：训练结束后在 10000 张测试图上计算总体准确率与**每个类别**的准确率，打印成表并保存到 outputs/test_predictions.npz（含 y_true、y_pred）。

你可以通过命令行改变不同的参数值训练模型。

训练结束后，你可以通过运行以下的指令观察测试的图片预测情况

```
python show_predict_picture.py
```

你会看到类似的结果，默认随机显示出12张测试集中图片的真实和预测标签，便于你直观的观察。

Random Samples (12 images)

Accuracy: 7/12 = 58.33%

Pred: airplane
True: cat



Pred: airplane
True: airplane



Pred: bird
True: deer



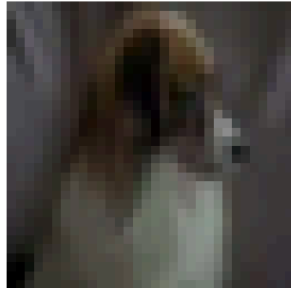
Pred: bird
True: bird



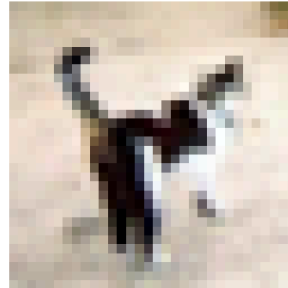
Pred: airplane
True: airplane



Pred: cat
True: dog



Pred: airplane
True: cat



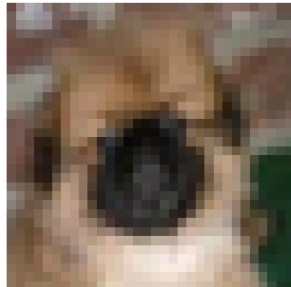
Pred: airplane
True: airplane



Pred: bird
True: bird



Pred: dog
True: dog



Pred: ship
True: airplane



Pred: deer
True: deer



你在实验报告里面需要写什么？

一. 请完成三组对照实验并在报告中分析：

优化器对比：固定其他超参数，分别用 `sgd`（需调大 `lr`）、`sgd_momentum`、`adam` 训练相同 `epoch` 数，对比损失曲线与验证准确率。

深度/宽度：尝试 `--num-blocks 3`（注意展平维度变化）或 `--base-filters 48` 等深度宽度的变化，观察准确率与训练时间的权衡。

预处理对比：仅 `[0,1]` 缩放 vs. 减均值 vs. 按通道标准化（代码初始默认减均值），记录初始损失与收敛速度差异。预处理部分已在 `train_cifar10.py` 的注释部分给出，选择对应的注释切换即可。

二. 思考以下问题，在报告中给出回答：

1. 多次运行 `show_predict_picture.py`，查看一组预测的结果，你发现哪些类别预测效果较好，哪些较差，你觉得最有可能的原因是什么？
2. 如果你观察到训练准确率持续上升，但验证准确率在第 5 个 `epoch` 后停滞或下降，这说明什么？在你的实验中是否出现了这种现象？如果出现过拟合，你会如何调整超参数来缓解？
3. 综合三组实验，优化器类型、模型深度/宽度、预处理方式这三个因素中，哪一个对最终测试准确率的影响最大？请提供具体数据支持你的结论。

最终报告内容提交：三组对比的实验的结果和分析（图文并茂），思考题，提交最终的test_predictions.npz并在报告中注明得到该预测结果所使用的完整命令行（含所有参数），以便助教复现，实验报告无模版、格式不限。

提示：纯 NumPy 的 CNN 不快。建议先用 `--epochs 1` 跑通全流程确认无误，再做完整实验。

提供的代码（无需修改）

以下文件已经写好，直接使用：

文件	内容
src/optim.py	sgd / sgd_momentum / adam
src/init.py	he_conv：卷积权重的 He 初始化
src/data.py	CIFAR-10 下载、加载、缓存、划分与 minibatch 采样
tests/gradient_check.py	数值梯度检验工具
train_cifar10.py	Task 3 的命令行训练脚本，进行预处理对比实验需要更改初始化方式
show_predict_picture.py	一组测试集的展示

评分标准

本次实验满分 **100 分**，由「自动化测试（80 分）」+「CIFAR-10 训练与报告（20 分）」两部分组成。两部分相互独立、分别打分后相加。

第一部分：自动化测试（80 分）

由 `uv run pytest` 共 47 例的通过比例决定：

$$\text{本部分得分} = \frac{\text{通过用例数}}{47} \times 80$$

其中 3 个测试无需写任何代码即可通过（仅检查已提供的 `TrainConfig` 与 `ConvNet.__init__`）。**请假定助教不会逐行阅读你的源代码，仅通过运行测试用例计算这部分分数。但会对代码进行抽查。**不得针对测试用例硬编码、或以其他方式绕过计算。

第二部分：CIFAR-10 训练与报告（20 分）

这部分考察你是否真正把模型在真实数据上「跑起来、跑对、能看懂」，评分依据为提交的报告 + 运行产物。
得分要点：

- 1.完成全部三组对照实验，分析基本正确（12分）
- 2.回答思考题，思路基本正确（6分）
- 3.报告书写规范，图文并茂，逻辑清晰，结果可验证（2分）

提交要求

请按要求完成实验代码，之后：

1. 将根目录下的 `.venv`、`.pytest_cache`、`data` 目录删除；
2. 将代码根目录和实验报告以 zip 格式打包提交，压缩包命名为 学号-姓名；

注意以下几点：

- 实验过程中，除了 `uv` 自动生成的文件，你只能修改 `src` 下的源文件（`train_cifar10.py` 为本次提供的运行脚本，属可修改范围）；
- 第一部分的自动化测试得分仅由 `uv run pytest` 决定；第二部分的 20 分由助教依据上述标准人工评定。

提交 ddl：2026 年 10 月 14 日。

参考资料

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf> LeNet-5 的论文，卷积网络的奠基工作。第 2 节对局部感受野、权重共享与下采样三个核心思想的论述至今仍是最清晰的表述之一。
- [2] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS*. https://papers.nips.cc/paper_files/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks AlexNet，使卷积网络成为主流的工作。
- [3] Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. *Technical Report*, University of Toronto. <https://www.cs.toronto.edu/~kriz/cifar.html> CIFAR-10 数据集的原始出处。
- [4] Karpathy, A., et al. *CS231n: Deep Learning for Computer Vision*, Stanford University. <https://cs231n.github.io/> 本次实验的接口约定完全沿用该课程：输入 x 形状为 (N, C, H, W) ，卷积核 w 形状为 (F, C, HH, WW) ，`conv_param` 包含 `stride` 与 `pad`，`pool_param` 包含 `pool_height`、`pool_width` 与 `stride`。课程笔记 [Convolutional Networks](#) 详细解释了输出尺寸公式与感受野的概念。
- [5] Johnson, J. Derivatives, Backpropagation, and Vectorization. *CS231n Course Notes*, Stanford University. <https://cs231n.stanford.edu/handouts/derivatives.pdf> 矩阵形式反向传播的推导方法。
- [6] Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*. <https://arxiv.org/abs/1409.1556> VGG 的论文。第 2.3 节论证了「两个 3×3 卷积的感受野等同于一个 5×5 卷积，但参数更少且非线性更强」，对应本实验中 `test_receptive_field_stacked_3x3_equals_one_5x5` 这一测试。
- [7] 斋藤康毅. (2016). *ゼロから作る Deep Learning*. O'Reilly Japan.（中译《深度学习入门：基于 Python 的理论与实现》，人民邮电出版社）本实验的 `im2col / col2im` 采用该书第 7 章的实现方式：只对卷积核位置做循环（ $HH \times WW$ 次），批次与空间位置全部通过带步长的切片处理。配套代码：<https://github.com/oreilly-japan/deep-learning-from-scratch>
- [8] Dumoulin, V., & Visin, F. (2016). A Guide to Convolution Arithmetic for Deep Learning. *arXiv:1603.07285*. <https://arxiv.org/abs/1603.07285> 系统整理了卷积、转置卷积、空洞卷积的尺寸计算规则，配有大量动画示意图，是理清 `stride` 与 `padding` 关系的最佳参考。
- [9] Araujo, A., Norris, W., & Sim, J. (2019). Computing Receptive Fields of Convolutional Neural Networks. *Distill*. <https://distill.pub/2019/computing-receptive-fields/> 感受野计算的完整推导，包含本实验中使用的递推公式及其在更复杂结构（如残差连接）下的推广。
- [10] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539> 深度学习的综述，其中对卷积网络在视觉任务中归纳偏置的讨论与本实验动机直接相关。